MEGN 651A: Homework Three

Rachel Bertaud

Feburary 24th, 2026

The code used to produce this work can be located at this address:

https://github.com/rachbertaud/Advanced_CFD/tree/main/03_steady_advection_diffusion.

For reproduction of results, download the folder `03_steady_advection_diffusion` and enter the directory of the corresponding case you would like to reproduce. Simply run `$ make build` and `$ make run` to produce the main data files. To produce error data for all the cases besides main, run `$ make ErrorData`. For visualization, run `viz.m` in the respective folder to produce and save the plots provided below. For the error plots, run `errorplot.m`. Here are the following case types:

- `Diff_Check` - Verifies pure diffusion solver
- `Neumann_Implementation` - Verifies implementation of Neumann Boundary Conditions
- `Verf_of_Main` - Verifies inclusion of advection to solver (verifies main)
- `Main` - Runs the physical example we used our solver on

`$ make clean` removes all the executables and all the `txt` files in the respective folders.

Overview of Code

The purpose of this code is to numerically solve the steady, two-dimensional advection-diffusion equation at discrete locations using a finite-volume method. Our goal is to simulate the temperature distribution of air moved by a ceiling fan with insulated walls and fixed temperature.

Before we get there, we must consider the following governing equation:

$$\int_{CS} (\rho c_p \mathbf{u}\phi) \cdot \mathbf{n}\, dA = \int_{CS} k\,(\nabla\phi \cdot \mathbf{n})\, dA + \int_{CV} f\, d\mathcal{V},$$

where k is the thermal conductivity (W/m·K), ρ is the density (kg/m^3), and c_p is the specific heat capacity (J/kg·K).

This can be discretized as

$$(A_P + D_P)\phi_P = (A_E + D_E)\phi_E + (A_W + D_W)\phi_W + (A_N + D_N)\phi_N + (A_S + D_S)\phi_S + b, \tag{1}$$

where the source term is

$$b = f_P \Delta x \Delta y,$$

the diffusion coefficients are

$$D_E = \frac{\Gamma_e \Delta y}{x_E - x_P}, \quad D_W = \frac{\Gamma_w \Delta y}{x_P - x_W}, \quad D_N = \frac{\Gamma_n \Delta x}{y_N - y_P}, \quad D_S = \frac{\Gamma_s \Delta x}{y_P - y_S},$$

$$D_P = D_E + D_W + D_N + D_S,$$

and the advection coefficients are

$$A_E = -F_e\sigma_e\Delta y, \quad A_W = F_w(1-\sigma_w)\Delta y, \quad A_N = -F_n\sigma_n\Delta x, \quad A_S = F_s(1-\sigma_s)\Delta x,$$

$$A_P = F_e(1-\sigma_e)\Delta y - F_w\sigma_w\Delta y + F_n(1-\sigma_n)\Delta x - F_s\sigma_s\Delta x.$$

We further define the follow

$$\sigma_e = \frac{x_e - x_P}{x_E - x_P}, \quad \sigma_w = \frac{x_w - x_W}{x_P - x_W}, \quad \sigma_n = \frac{y_n - y_P}{y_N - y_P}, \quad \sigma_s = \frac{y_s - y_S}{y_P - y_S}.$$

Note that we will analytically solve for $c(x, y)$ using linear algebra. We solve

$$\mathbf{Ac} = \mathbf{x}$$

for $\mathbf{c}$ given $\mathbf{x}$ and $\mathbf{A}$, which are built using the discretized equations above.

Verification

Our overall goal here is to verify the designed code works as anticipated. We first define our manufactured solution for the verification steps as

$$c_m(x, y) = \cos(x)\cos(y).$$

Pure Diffusion

First, we discretize as defined for the pure diffusion case (without the advection term) with fully Dirichlet boundary conditions. Note for this case that

$$f_p(x, y) = -2\cos(x)\cos(y).$$

We then build $\mathbf{A}$ and $\mathbf{b}$, and solve for $\mathbf{c}$. Using the results of this step, we verify that

1. Our solution's relative error measured with the infinity norm converges with order two accuracy and
2. Our operator, $\mathbf{A}$, computes the expected solution for pure diffusion.

Neumann Conditions on W/E Boundaries

Second, we discretize as defined for the pure diffusion case (without the advection term) with Dirichlet boundary conditions on the N/S boundaries and the following Neumann conditions:

$$\left.\frac{\partial T}{\partial x}\right|_{x=0,Lx} = -\sin(x)\cos(y).$$

Note for this case that

$$f_p(x, y) = -2\cos(x)\cos(y).$$

We then build $\mathbf{A}$ and $\mathbf{b}$, and solve for $\mathbf{c}$. Using the results of this step, we verify that

1. Our solution's relative error measured with the infinity norm converges with order two accuracy and
2. Our operator, $\mathbf{A}$, computes the expected solution for pure diffusion with the addition of Neumann boundary conditions on the W/E boundaries.

Verification of Main

Finally, we discretize as defined for the steady, 2D advection-diffusion equation with Dirichlet boundary conditions on the N/S boundaries and the following Neumann conditions:

$$\left.\frac{\partial T}{\partial x}\right|_{x=0,Lx} = -\sin(x)\cos(y).$$

Note for this case we must define the following:

$$f_p(x,y) = -k\nabla^2 T_e + \rho c_p\left(u\frac{\partial T_e}{\partial x} + v\frac{\partial T_e}{\partial y}\right)$$

$$u(x,y) = -\frac{A}{b_y L_y}\sin\left(\frac{2\pi x}{b_x L_x}\right)\cos\left(\frac{2\pi y}{b_y L_y}\right) \text{ m/s},$$

$$v(x,y) = \frac{A}{b_x L_x}\cos\left(\frac{2\pi x}{b_x L_x}\right)\sin\left(\frac{2\pi y}{b_y L_y}\right) \text{ m/s},$$

where u and v are the velocity components (m/s), A is the flow amplitude (m^2/s), and b_x, b_y, L_x, L_y are prescribed dimensionless constants that determine the spatial structure of the flow and dimensional length scales (m), respectively. We define $T_e(x) = \cos(x)\cos(y)$.

We then build $\mathbf{A}$ and $\mathbf{b}$ as explicitly defined in the overview section, and solve for $\mathbf{c}$. Using the results of this step, we verify that

1. Our solution's relative error measured with the infinity norm converges with order two accuracy and
2. Our operator, $\mathbf{A}$, computes the expected solution for the 2D steady advection diffusion equation with the defined boundary conditions.

The results of the verification process are defined below in the "Results" section.

Ceiling Fan Application

Upon verification, we study the following problem. Consider a domain defined as $x \in [0, L_x]$ and $y \in [0, L_y]$. We will consider the effect of advection generated by a ceiling fan, which drives the incompressible velocity field

$$u(x,y) = -\frac{A}{b_y L_y}\sin\left(\frac{2\pi x}{b_x L_x}\right)\cos\left(\frac{2\pi y}{b_y L_y}\right) \text{ m/s},$$

$$v(x,y) = \frac{A}{b_x L_x}\cos\left(\frac{2\pi x}{b_x L_x}\right)\sin\left(\frac{2\pi y}{b_y L_y}\right) \text{ m/s}.$$

Note that this is the same velocity field that we used for the verification of main case. We solve for the function of temperature distribution, $T(x,y)$, of the fluid caused by advection and diffusion terms and remove the source terms.

We define our boundary conditions as follows:

$$T\Big|_{y=0} = 280 \text{ K}, \quad T\Big|_{x=Ly} = 300, \text{ K} \quad \left.\frac{\partial T}{\partial x}\right|_{x=0,Lx} = -\sin(x)\cos(y).$$

Error Analysis

During the verification process, error of the numerical solution is calculated as relative error using the infinity norm, which is defined as

$$E_\infty = \frac{||c_{\text{numerical}} - c_{\text{analytical}}||_\infty}{||c_{\text{analytical}}||_\infty} = \frac{\max|c_{\text{numerical}} - c_{\text{analytical}}|}{\max|c_{\text{analytical}}|}.$$

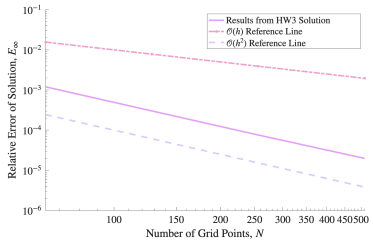
Note that we will also define local error as

$$\text{Local Error} = |c_{\text{numerical}} - c_{\text{analytical}}|.$$

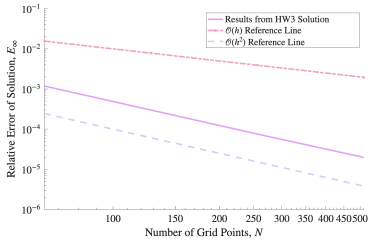
Results

Overall, the results of this code verified its capabilities and produced a intuitive solution to the fuel cell application problem. Everything performed as anticipated. Refer to

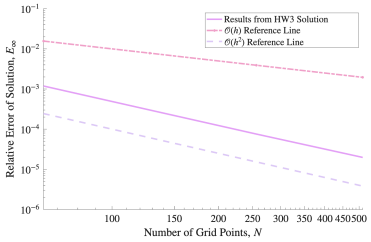
- Figure 1 for the error plots for the different verification cases.
- Figure 2 for the results of the verification with pure diffusion and full Dirichlet BC.
- Figure 3 for the results verification of pure diffusion with Dirichlet BC on N/S and Neumann on E/W.
- Figure 4 for the results verification of steady advection-diffusion with Dirichlet BC on N/S and Neumann on E/W.
- Figure 5 for the requested results of the ceiling fan application problem.



(a) Pure diffusivity with full Dirichlet.



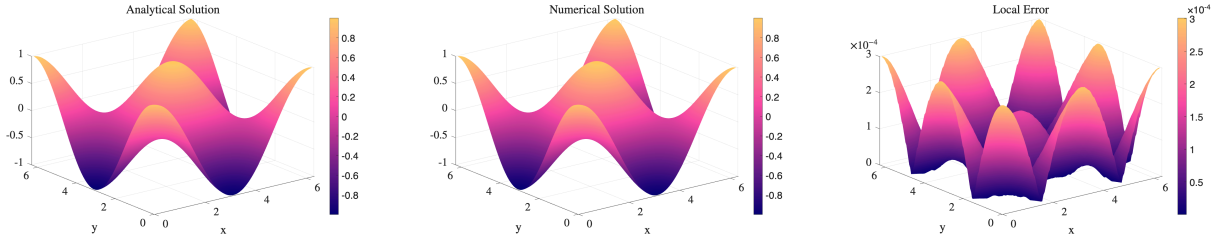
(b) Pure diffusivity with Dirichlet BC on N/S and Neumann on E/W.



(c) Steady advection-diffusion with Dirichlet BC on N/S and Neumann on E/W.

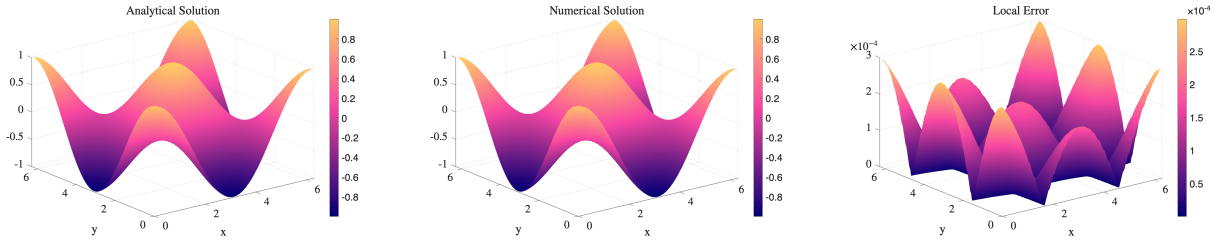
Figure 1: Relative error versus number of grid points for verification cases. All cases converge with second-order accuracy, which is what we anticipate.

Below are the visual solutions of our simulation alongside the analytical solution for $L_x = L_y = 2\pi$ & $N_x = N_y = 128$. We can be confident in the accuracy of the code from the local error plot, noting higher errors near the boundaries.



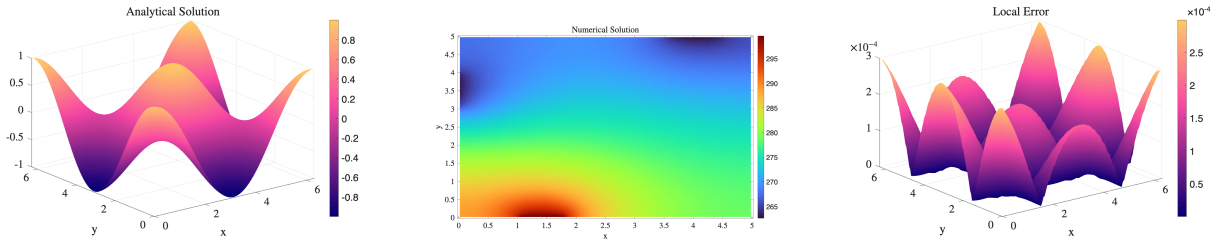
(a) Analytical Solution (b) Numerical Solution (c) Local Error

Figure 2: Results of the simulation code with with pure diffusion and full Dirichlet BC.



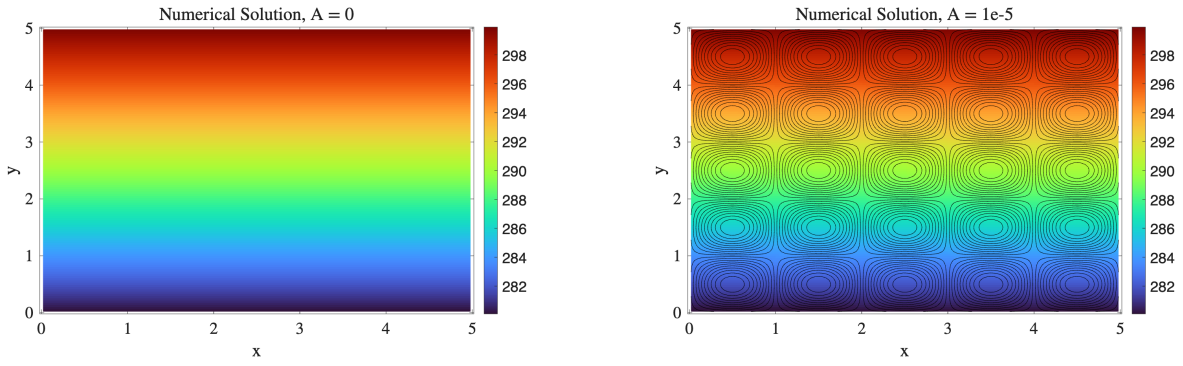
(a) Analytical Solution (b) Numerical Solution (c) Local Error

Figure 3: Results of the simulation code with pure diffusion with Dirichlet BC on N/S and Neumann on E/W.



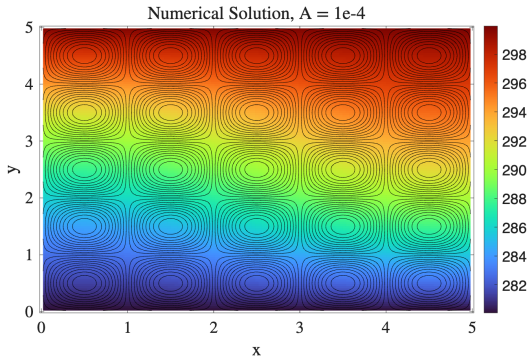
(a) Analytical Solution (b) Numerical Solution (c) Local Error

Figure 4: Results of the simulation code with steady advection-diffusion with Dirichlet BC on N/S and Neumann on E/W.

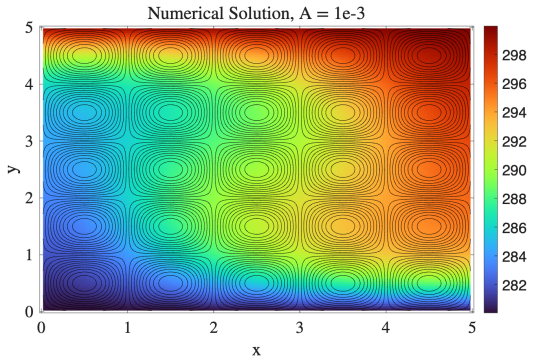


(a) $A = 0$

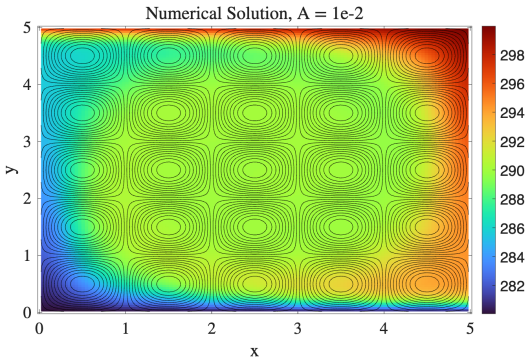
(b) $A = 10^{-5}$



(c) $A = 10^{-4}$



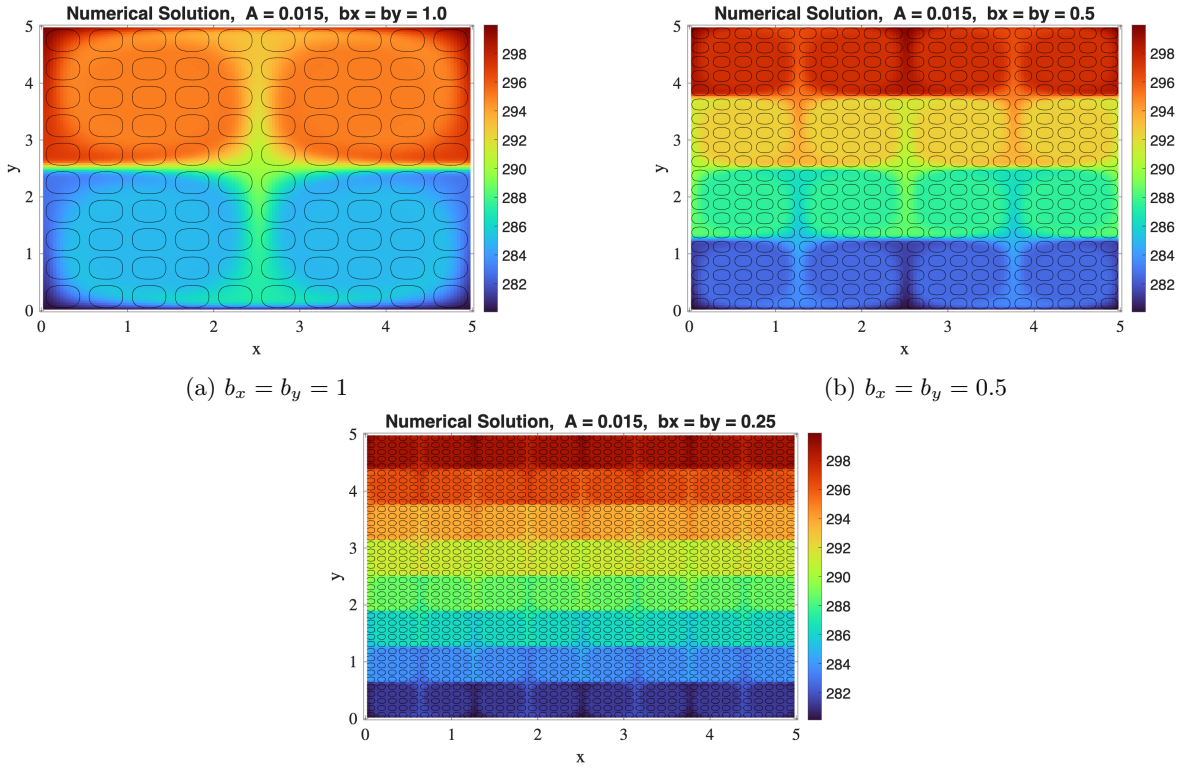
(d) $A = 10^{-3}$



(e) $A = 10^{-2}$

Figure 5: Numerical solution for $b_x = b_y = 2$, varying flow amplitude A.

As we look at these results, we see that as we increase A from 0 to 10^{-2}, pure diffusion ($A = 0$) goes to what we would expect to see from a ceiling fan. The fan whips up the cool air on the left boundary and drags down the hot air on the right. Recall that these temperatures are enforced by our boundary conditions. This parameter sweeps helps us see the impact of advection on the temperature field while diffusion is being applied.



(a) $b_x = b_y = 1$

(b) $b_x = b_y = 0.5$

(c) $b_x = b_y = 0.25$

Figure 6: Numerical solution for $A = 0.015$, varying length scales $b_x = b_y$.

The main take away from these results show that diffusion dominates all three of these cases. We can consider the Péclet number, defined as

$$\text{Pe} = \frac{UL}{\mathcal{D}} = \frac{UL}{k}$$

where U and L are the characteristic velocity and length of the simulation. For all three of these cases, the Péclet number is relatively small, as $A = 0.015$, which means we would see diffusion dominate. As we make $bx = by$ smaller, we see diffusion dominate even more, which follows the Péclet number definition, as we decrease the characteristic velocity.

We can also consider the number of vorticies that appear. As we decrease $bx = by$, we decrease the period of the streamfunction. For $bx = by = 1$, the period would be 5, which matches the number of vorticies we see. As we halve $bx = by$, we see the number of voritices double in each dimension respectively.

Code

For the sake of page usage, I am only including the code that builds the LHS/RHS and the main code that runs the solver. Everything else that was used to produce these results can be found at this GitHub location. Please refer to the first page for reproduction instructions.

main.cpp

```cpp
#include <iostream> // needed for cout
#include "Header_Case.h" // needed for the functions
#include "Header_Mesh.h"
#include "Header_Build.h"
#include "Header_EigenSolve.h"
#include "Header_Adv.h"

int main()
{

        caseType Case;          //declare case
        readCase(Case);
        dispCase(Case);

        meshType Mesh;
        readMesh(Mesh);
        dispMesh(Mesh);
        createMesh(Mesh);
        writeMesh(Mesh);

        matrixType A;
        matrixType RHS;

        A.init(Mesh);
        RHS.init(Mesh);

        surfaceScalarField Flux;
        Flux.allocate(Mesh);
        ComputePhix(Flux, Mesh, Case);
        ComputePhiy(Flux, Mesh, Case);

        BuildA(A, Mesh, Case, Flux);
        BuildRHS(RHS, Mesh, Case);

        int flag = EigenSolve(A, RHS);

        return flag;
}
```

funcs_Build.cpp

```cpp
#include <iostream>             //for cout
#include <vector>
#include <cmath>

#include "Header_Build.h"       // for builder funcs/structs
#include "Header_Mesh.h"        // for mesh funcs/structs
#include "Header_Case.h"        // for case funcs/structs
#include "Header_Adv.h"         // for advection funcs/structs
```

```
9
10 void matrixType::init(const meshType& Mesh)
11 {
12         Nx = Mesh.Nx;
13         Ny = Mesh.Ny;
14         Nx2 = Nx + 2;
15         Ny2 = Ny + 2;
16         B.assign(Nx2*Ny2, 0.0);
17 }
18
19 void matrixType::addEntry(int row, int col, double val)
20 {
21         ROW.push_back(row);
22         COL.push_back(col);
23         VAL.push_back(val);
24 }
25
26 void matrixType::setRHS(int i, int j, double val)
27 {
28         B[i + j*Nx2] = val;
29 }
30
31 double matrixType::getRHS(int i, int j) const
32 {
33         return B[i + j*Nx2];
34 }
35
36 void BuildA(matrixType& Matrix,
37             const meshType& Mesh,
38             const caseType& Case,
39             const surfaceScalarField& Flux)
40 {
41         int Nx = Matrix.Nx;
42         int Ny = Matrix.Ny;
43         int Nx2 = Matrix.Nx2;
44         int Ny2 = Matrix.Ny2;
45         double dx = Mesh.dx;
46         double dy = Mesh.dy;
47         double k = Case.k;
48
49         int row;
50
51         //south/north boundary conditions (Dirichlet)
52         for (int i = 1; i <= Nx; i++)
53         {
54                 // south
55                 row = i;
56                 Matrix.addEntry(row, row , 0.5);
57                 Matrix.addEntry(row, row + Nx2, 0.5);
58
59                 //north
60                 row = i + (Nx2*(Ny + 1));
61                 Matrix.addEntry(row, row, 0.5);
62                 Matrix.addEntry(row, row - Nx2, 0.5);
63         }
64
65         //east/west boundary conditions (Neumann)
66         for (int j = 1; j <= Ny; j++)
67         {
```

```
68                  //west
69                  row = Nx2*j;
70                  Matrix.addEntry(row, row, -1/dx);
71                  Matrix.addEntry(row, row + 1, 1/dx);
72
73                  //east
74                  row = Nx2*(j + 1) - 1;
75                  Matrix.addEntry(row, row, 1/dx);
76                  Matrix.addEntry(row, row - 1, -1/dx);
77          }
78
79          //five-point stencil
80          double De  = k*dy/dx;
81          double Dw = k*dy/dx;
82          double Ds = k*dx/dy;
83          double Dn = k*dx/dy;
84          double Dp = De + Dw + Ds + Dn;
85
86
87          double Se = 0.5;
88          double Sw = 0.5;
89          double Sn = 0.5;
90          double Ss = 0.5;
91
92
93          double cp = Case.cp;
94
95          //internal nodes of A
96
97          for (int i = 1; i <= Nx; i++)
98          {
99                  for (int j = 1; j <= Ny; j++)
100                 {
101
102                         double Fe = Flux.gete(i,j);
103                         double Fw = Flux.getw(i,j);
104                         double Fs = Flux.gets(i,j);
105                         double Fn = Flux.getn(i,j);
106
107                         double Ae = -Fe*Se*dy;
108                         double Aw = Fw*(1 - Sw)*dy;
109                         double As = Fs*(1 - Ss)*dx;
110                         double An = -Fn*Sn*dx;
111                         double Ap = Fe*(1 - Se)*dy - Fw*Sw*dy + Fn*(1 - Sn)*dx -
                                Fs*Ss*dx;
112
113                         double ae = De + Ae;
114                         double aw = Dw + Aw;
115                         double as = Ds + As;
116                         double an = Dn + An;
117                         double ap = Dp + Ap;
118
119                         row = Nx2*j + i;
120
121                         Matrix.addEntry(row, row, ap);
122                         Matrix.addEntry(row, row + 1, -ae);
123                         Matrix.addEntry(row, row - 1, -aw);
124                         Matrix.addEntry(row, row + Nx2, -an);
125                         Matrix.addEntry(row, row - Nx2, -as);
```

```
126                 }
127         }
128
129         // extraneous nodes
130
131         //bottom left
132         Matrix.addEntry(0,0,1);
133
134         //bottom right
135         Matrix.addEntry(Nx2 - 1, Nx2 - 1, 1);
136
137         //top left
138         Matrix.addEntry(Nx2*(Ny + 1), Nx2*(Ny + 1), 1);
139
140         //top right
141         Matrix.addEntry((Nx2*Ny2 - 1), (Nx2*Ny2 - 1), 1);
142
143 }
144
145 void BuildRHS(matrixType& Matrix,
146               const meshType& Mesh,
147               const caseType& Case)
148 {
149         int Nx = Matrix.Nx;
150         int Ny = Matrix.Ny;
151         int Nx2 = Matrix.Nx2;
152         int Ny2 = Matrix.Ny2;
153
154         double dx = Mesh.dx;
155         double dy = Mesh.dy;
156         double Lx = Mesh.Lx;
157         double Ly = Mesh.Ly;
158
159         double k = Case.k;
160         double by = Case.by;
161         double bx = Case.bx;
162
163         // south/north BC (Dirichlet)
164         for (int i = 1; i <= Nx; i++)
165         {
166                 //south
167                 double gs = Case.Ts;
168                 Matrix.setRHS(i, 0, gs);
169
170                 //north
171                 double gn = Case.Tn;
172                 Matrix.setRHS(i, Ny + 1, gn);
173         }
174
175         // east/west BC (Neumann)
176         for (int j = 1; j <= Ny; j++)
177         {
178                 //west
179                 double gw = 0;
180                 Matrix.setRHS(0, j, gw);
181
182                 //east
183                 double ge = 0;
184                 Matrix.setRHS((Nx + 1), j, ge);
```

```
185             }
186
187 }
```